Note sulla soluzione della prova scritta del 31 gennaio 2025

(a) Spiegate come il problema possa essere descritto e formalizzato in termini di grafi e come possa essere risolto adattando uno degli algoritmi visti a lezione (indicate quale).

Come indicato nella parte del testo precedente, il grafo rappresenta la mappa della metropolitana: i vertici rappresentano le stazioni, gli archi rappresentano i tratti che collegano direttamente due stazioni (cioè senza stazioni intermedie).
Si presti attenzione al fatto che i *percorsi* (descritti all'inizio del testo) sono *cammini* nel grafo. Quindi gli archi del grafo non sono i percorsi.
Le tariffe corrispondono ai pesi dei cammini minimi in un grafo orientato in cui un arco orientato (u,v) corrisponde all'arco (non orientato) (u,v) del grafo originario e ha peso c(v) (ad ogni arco del grafo originario corrispondono due archi orientati).

(b) Supponete che vi siano n stazioni, indicate con $1, 2, \ldots, n$. Scrivete lo pseudocodice di un algoritmo che risolva il problema. L'algoritmo deve costruire una matrice M dove l'elemento $M_{i,j}$ indica la tariffa da pagare se si entra nella stazione i e si esce dalla stazione j. L'algoritmo deve restituire la coppia di stazioni $(\texttt{xMax}, \texttt{yMax})$ per la quale la tariffa da `xMax` a `yMax` risulta massima (nel caso vi sia più di una coppia per la quale la tariffa è massima, l'algoritmo ne restituisce una qualunque).

Lo pseudocodice deve rispettare le specifiche fornite. In particolare, come indicato nella parte del testo precedente e riportata qui sotto, l'algoritmo deve ricevere in ingresso il grafo e la funzione c (dunque non grafi o funzioni differenti o altro):

- riceva in ingresso il grafo non orientato $G = (V, E)$ che rappresenta la mappa della metropolitana e la funzione $c : V \to \mathbb{N}$ che associa a ogni stazione il costo,
- restituisca la coppia di stazioni di ingresso e uscita per le quali la tariffa da pagare risulta massima.

Soluzioni che ricevano in ingresso informazioni differenti da quelle specificate nel testo dell'esercizio, NON possono essere accettate. In particolare, nel testo dell'esercizio non è definita alcuna funzione chiamata w

È sufficiente modificare la parte iniziale dell'algoritmo di Floyd-Warshall, assegnando all'elemento di riga *i* e colonna *j* della matrice il valore *c(j)* nel caso *i≠j* e vi sia l'arco che congiunge i vertici di indici *i* e *j*. Una volta calcolata la matrice M delle tariffe, si ricava la posizione del valore massimo.

Attenzione: durante l'esecuzione dell'algoritmo di Floyd-Warshall, le distanze nella matrice M sono "provvisorie" e possono essere diminuite nelle iterazioni successive. Pertanto il massimo deve essere cercato solo quando è stato completato il calcolo di M, cioè al termine dell'algoritmo di Floyd-Warshall e non prima!

(c) Fornite una stima del tempo di calcolo dell'algoritmo ottenuto, in funzione del numero n di stazioni, *giustificando adeguatamente la risposta.*

La risposta deve essere giustificata facendo riferimento allo pseudocodice scritto. Non è sufficiente guardare i cicli e il numero di iterazioni: occorre considerare quali e quante operazioni contengono.

(d) Spiegate come modificare o integrare l'algoritmo precedente nel caso si debba ricavare un elenco di *tutte le coppie* di stazioni $(\mathtt{x}, \mathtt{y})$ per le quali la tariffa da $\mathtt{x}$ a $\mathtt{y}$ è massima. Scrivete poi lo pseudocodice di questa parte.

Nella parte precedente si può memorizzare, oltre alla posizione del massimo, il suo valore. È sufficiente poi scorrere la matrice M e, ogni volta che si trova il valore massimo, se ne memorizzano gli indici nella *prima posizione libera* di un array (di grandezza n^2) o all'*inizio* di una lista.

Soluzioni in cui si costruisce l'elenco prima di avere individuato il valore massimo, memorizzando man mano nell'elenco le posizioni del massimo sinora trovato e svuotando l'elenco quando si aggiorna il valore del massimo sono del tutto discutibili (per non dire disdicevoli). Che senso ha memorizzare dei dati che poi verranno buttati? C'è un inutile spreco di spazio e di tempo (anche se asintoticamente la complessità potrebbe essere la stessa della soluzione indicata sopra).

(e) Fornite una stima del tempo di calcolo della parte di algoritmo *al punto precedente*, in funzione del numero n di stazioni, giustificando adeguatamente la risposta.

È richiesto il costo solo della parte al punto (d), non di tutto l'algoritmo!

Scandire la matrice e trovare il massimo richiede un numero quadratico di passi in n. Se si inseriscono le posizioni trovate in una struttura dove il costo dell'inserimento è O(1) (esempio a inizio lista oppure in un array, riempito a partire dalla posizione 0, ricordando man mano l'indice della prima posizione libera) allora il costo di questa parte è O(n^2). Se si utilizzano strutture con inserimenti più costosi, il costo totale sarà più elevato.

(f) Supponete che *al posto di tutto ciò che è richiesto al punto (b)*, si debba costruire un algoritmo che restituisca una coppia di stazioni (`xMin`, `yMin`) per le quali la tariffa da `xMin` a `yMin` risulti minima (escludendo le coppie in cui la stazione di uscita coincide con quella di ingresso). È possibile ottenere un algoritmo più efficiente in termini di tempo di quello ottenuto al punto (b)? E in termini di spazio?

Poiché non ci sono costi negativi, il costo minimo corrisponderà al costo di un arco di peso minimo cioè, rispetto alla specifica data, di un arco che entra in un vertice x con c(x) minima.
Non è dunque necessario calcolare e memorizzare la matrice M: questo porta a una riduzione del tempo di calcolo e dello spazio utilizzato dall'algoritmo.
Ci sono varie soluzioni, ad esempio si può trovare un vertice x con c(x) minima e un arco (y,x), oppure esaminare direttamente gli archi del grafo (i dettagli dipendono dalla rappresentazione del grafo).
In ogni caso, poiché il numero di archi è al più n^2, si può trovare facilmente una soluzione che usa tempo O(n^2).
La descrizione del grafo è data come input. Lo spazio aggiuntivo rispetto ad essa è costante (anziché O(n^2) calcolando e memorizzando M).